

Universidade Federal do Rio Grande do Sul



INF01151 - Sistemas Operacionais II N
Prof. Eder John Scheid

Relatório Trabalho Prático

Etapa 1

Gabriel Luz Fallavena
Kelvin Schaun Brose
Sandro Viegas
Turma B

26 de Outubro de 2025

Descrição do Relatório

A equipe deverá produzir um relatório fornecendo os seguintes dados:

- Explicação e respectivas justificativas a respeito de:

1. Como foi a implementação cada subserviço

Subserviços

O projeto foi dividido em dois subserviços principais: Discovery e Pix Request, cada um com funções específicas:

DiscoveryService permite que os clientes descubram o servidor disponível na rede sem conhecer previamente o IP. Na sua implementação foram utilizados, para os clientes, o pacote UDP broadcast (PACKET_TYPE_DISCOVERY) para a porta descoberta. E o servidor recebe o pacote, registrando os clientes no banco de dados caso seja novo, e envia uma resposta de confirmação (PACKET_TYPE_DISCOVERY_ACK).

Resposta com ACK garante que o cliente confirme que foi registrado no servidor.

O broadcast UDP permite descobrir o servidor dinamicamente, sendo bom para rede locais onde o IP de servidor pode variar.

O segundo subserviço é o PixService cujo objetivo é realizar as transferências de valores e consultar o saldo. Na implementação o cliente cria requisições do tipo PACKET_TYPE_REQUEST, aguarda o ACK, e realiza até 3 tentativas caso falha. O Servidor processa as requisições, verificando validade, atualiza o saldo e envia o ACK para o cliente com o saldo atualizado.

A utilização do ACK garante confiabilidade para o sistema não confiável do UDP.

2. Em quais áreas do código foi necessário garantir sincronização no acesso a dados

Sincronização

Como o servidor processa múltiplas requisições de forma concorrente, é necessário garantir acesso seguro ao banco de dados compartilhado.

Banco de dados de clientes, que é utilizado em *procesarRequisicao* e *handle_discovery*, ele é protegido por mutex para evitar condição de corrida durante a leitura e escrita de saldos e registro de novos clientes.

Variáveis globais, são variáveis que é afetada por múltiplos clientes, *num_transactions*, *total_transferred* e *total_balance*, e são atualizado no mesmo lock para garantir consistência.

E sem estas sincronizações, duas threads poderiam atualizar saldos simultaneamente, causando valores incorretos ou perda de transações, e pelas especificações cada nova transferência é para ser uma thread.

3. Descrição das principais estruturas e funções que a equipe implementou

Estruturas

packet_t: estrutura base para encapsular mensagens enviadas entre cliente e servidor, incluindo tipo de pacote, sequência e dados.

request: contém informações de uma solicitação de transferência (endereço de destino e valor).

request_ack: representa o reconhecimento (ACK) de uma requisição processada com sucesso.

sockaddr_in: estrutura da API de sockets usada para definir endereços IP e portas.

Funções

Cliente:

`discover_server(port)`: envia broadcast, recebe ACK do servidor.

`send_request_with_retry(socket, server_ip, port, req, ack)`: envia requisições com retry.

`send_exit_request(socket, server_ip, port)`: notifica servidor para encerrar sessão.

`receive_request_ack(...)` → Recebe ACK e valida sequência.

Servidor

`processarRequisicao(value, ipRemetente, ipDestino)`: valida e aplica transações.

`handle_discovery(discoverySocket, clientAddr, seqn)`: registra clientes e responde ACK.

`handle_pix(serviceSocket, clientAddr, requisicao)`: processa Pix requests, envia ACK.

4. Explicar o uso das primitivas de comunicação inter-processos usadas

Primitivas de comunicação

`socket()` - cria um canal de comunicação bidirecional. No cliente é usado o socket para descoberta e para requisições. No server é usado tanto para mensagem de descoberta quanto as de requisições PIX.

`bind()` - associa o socket a um endereço de IP e uma porta específica. e utilizado apenas no server.

`sendto()` - O cliente envia pacotes de descobertas e requisições PIX ao servidor, e o servidor envia respostas ACK e mensagem de descoberta de volta aos clientes.

`recvfrom()` - O cliente aguarda respostas de descobertas e confirmações de requisições (ACK) do servidor. E o servidor recebe os pacotes de descobertas e requisições de PIX.

setsockopt() - usado apenas no cliente, configurando a opções no socket. é usado para definir um tempo máximo de espera (timeout) para receber o pacote de confirmação (ACK), para não ficar bloqueado indefinidamente.

select() - monitora múltiplos sockets e bloqueia até que algum esteja pronto para leitura ou escrita, permitindo que o servidor trate novas mensagem conforme chegam mantendo o controle de fluxo e a possibilidade de expansão futura para múltiplos sockets.

close() - fecha o socket e libera os recursos do sistemas.

• Descrição dos problemas que a equipe encontrou durante a implementação e como estes foram resolvidos (ou não)

Problemas encontrados

Um dos maiores problemas foi na integração entre cliente e servidor. Como o cliente e o servidor foram implementados por integrantes diferentes da equipe, surgiram dificuldades de compatibilidade entre as mensagens trocadas. As principais falhas ocorreram na fase inicial de comunicação, devido a diferenças no formato e tamanho dos pacotes (packet_t) e na ordem dos campos enviados. Isso resultava em falhas no reconhecimento de ACKs e respostas incorretas.

Foi realizada uma padronização da estrutura de pacotes (request, request_ack, packet_t) entre os dois lados, garantindo alinhamento de tipos e tamanhos (uint32_t, struct sockaddr_in, etc.). Também foram adicionados logs detalhados com timestamps para depuração da comunicação e validação do handshake inicial.

• Descrição de qual parte cada um dos integrantes do grupo atuou de forma ativa no desenvolvimento do trabalho

Atuação dos integrantes

A divisão de responsabilidades entre os integrantes do grupo foi definida da seguinte forma:

Sandro: Implementação completa da parte do cliente, incluindo envio de requisições, descoberta do servidor, retransmissão de pacotes e tratamento de ACKs.

Gabriel: Responsável principal pela parte do servidor, incluindo gerenciamento de clientes, processamento de transações PIX, sincronização de dados e manipulação de threads. Além disso, participou da realização dos testes do código.

Kelvin: Contribuiu para a implementação do servidor juntamente com Gabriel, focando em funções auxiliares, depuração e integração geral. Também participou da realização dos

testes do código e foi responsável por redigir inicialmente o relatório, que foi posteriormente revisado e incrementado por todos os integrantes.

A colaboração entre os membros garantiu que o sistema fosse implementado de forma modular, permitindo a integração eficiente entre cliente e servidor e a validação funcional do projeto por meio de testes.